

LABORATORIO DE ALGORITMOS Y ESTRUCTURAS DE DATOS

Unidad 5: Búsqueda, Ordenamiento e IA

Ordenamiento

Desafío 26 — Ordenamiento burbuja

El algoritmo de burbuja compara dos elementos adyacentes de una lista o array y los intercambia si están en el orden incorrecto. Repite este proceso de comparación e intercambio hasta que la estructura quede completamente ordenada.

Consigna:

Escribí un programa en Java que solicite al usuario ingresar 4 números enteros. Guardalos en una lista dinámica (`List<Integer>`), ordenalos utilizando el método de la burbuja y mostralos antes y después del ordenamiento. Además, debés medir el tiempo exacto que tarda en ejecutarse el algoritmo.

Para lograrlo, creá obligatoriamente las siguientes funciones:

- **`ingresarNumeros()`**: Solicita al usuario 4 números enteros por consola y los retorna en una lista (`List<Integer>`) completa. Si fuera necesario, retorná el tipo de dato que consideres.
- **`ordenarBurbuja(List<Integer> numeros)`**: Aplica el algoritmo de burbuja directamente sobre la lista recibida. Utiliza dos bucles `for` anidados para comparar e intercambiar los elementos adyacentes. Si fuera necesario, retorná el tipo de dato que consideres.
- **`mostrarLista(List<Integer> numeros)`**: Recorre la lista y muestra en pantalla cada elemento junto con su índice o posición. Si fuera necesario, retorná el tipo de dato que consideres.
- **`medirTiempo(List<Integer> numeros)`**: Captura el tiempo del sistema usando `System.nanoTime()` justo antes y después de invocar a `ordenarBurbuja()`. Calcula la diferencia y muestra el tiempo total de ejecución en nanosegundos. No es necesario retornar nada en este caso. Solo mostrá en pantalla el tiempo de ejecución en pantalla.

Desde el método `main()`, debés coordinar el flujo para mostrar la lista en su estado original, luego la lista ya ordenada y, finalmente, el tiempo que demoró el algoritmo en procesarla.

 *Saberes necesarios: for anidado, intercambio de variables con variable auxiliar, System.nanoTime()*

Desafío 27 — Ordenamiento por inserción

El algoritmo de inserción toma un elemento y lo inserta en la posición correcta dentro de la parte ya ordenada de la lista (o array). Es más eficiente que burbuja en la práctica.

Consigna

Usá la misma lista de 4 números del desafío anterior. Ordenalos ahora con inserción y medí el tiempo. Al final, comparalo con el tiempo que dio burbuja.

Creá las siguientes funciones:

- `ordenarInsercion(List numeros)` → aplicá el algoritmo de inserción. En cada iteración, tomá el elemento actual y mové los elementos mayores una posición hacia adelante hasta encontrar su lugar.
- `mostrarLista(List numeros)` → igual que en el desafío anterior.
- `medirTiempo(List numeros)` → igual que en el desafío anterior, pero para `ordenarInsercion()`.

Desde `main()`, mostrá la lista original, la ordenada, el tiempo de inserción y el tiempo de burbuja (guardalo de la corrida anterior). ¿Cuál fue más rápido?

💡 *Saberes necesarios: for + while, variable auxiliar para el elemento actual, `System.nanoTime()`. Comparación con burbuja: anotá el tiempo de cada uno y discutí el resultado con tu compañero.*

Búsqueda

Desafío 28 — Búsqueda lineal

La búsqueda lineal recorre un array (o lista) elemento por elemento hasta encontrar el valor buscado o llegar al final.

Consigna

Usá la misma lista de 4 números del desafío 26 (sin ordenar), pero ahora usala como array. Pedile al usuario un número a buscar. Indicá si lo encontraste y en qué posición. Medí el tiempo de búsqueda.

Creá las siguientes funciones:

- **`busquedaLineal(int[] numeros, int objetivo)`** → recorré el array con un `for`. Si encontrás el elemento, retorná su posición. Si no, retorná `-1`.
- **`mostrarResultado(int pos, int objetivo)`** → si `pos != -1`, mostrá en qué posición está. Si no, informá que no fue encontrado.
- **`medirTiempo(int[] numeros, int objetivo)`** → medí el tiempo de `busquedaLineal()` con `System.nanoTime()`.

💡 *Saberes necesarios: for, retorno de -1 como señal de "no encontrado" (igual que `indexOf` en listas), `System.nanoTime()`.*

Desafío 29 — Búsqueda binaria

La búsqueda binaria divide el array (o lista) por la mitad en cada paso. Es mucho más eficiente que la lineal, pero el array debe estar ordenado primero.

Consigna

Usá el array ya ordenado del desafío 27. Pedile al usuario un número a buscar. Aplicá búsqueda binaria, mostrá el resultado y medí el tiempo. Comparalo con el tiempo de búsqueda lineal.

Creá las siguientes funciones:

- **busquedaBinaria(int[] numeros, int objetivo)** → definí tres variables: `inicio`, `fin` y `medio`. Mientras `inicio <= fin`, calculá el medio y decidí si buscar a la izquierda o a la derecha. Retorná la posición o `-1` si no lo encontrás.
- **mostrarResultado(int pos, int objetivo)** → igual que en el desafío anterior.
- **medirTiempo(int[] numeros, int objetivo)** → medí el tiempo de `busquedaBinaria()`. Comparalo con el tiempo de búsqueda lineal. ¿Cuál fue más rápido?

 *Saberes necesarios: while, división de rangos (inicio/medio/fin), requisito de array ordenado. Recordá: sin ordenamiento previo, la búsqueda binaria da resultados incorrectos.*

Ejercicio integral

Desafío 30 — Búsqueda y ordenamiento con datos reales

Aplicá lo aprendido en los desafíos anteriores sobre datos reales: un registro de notas de alumnos.

El programa maneja dos listas paralelas: nombres de alumnos y sus notas. El usuario puede cargar datos, ordenarlos por nota y buscar la nota de un alumno específico.

Creá las siguientes funciones:

- Una función que **cargue datos** → pedile al usuario nombre y nota de cada alumno hasta que escriba "fin".
- Una función que **ordene por nota de mayor a menor** usando inserción. Cuando muevas una nota, mové también el nombre correspondiente para mantener las listas sincronizadas.
 - o Guardá la nota actual y el nombre actual en variables auxiliares.
 - o Recorré hacia la izquierda mientras la nota sea menor que la actual y mové los elementos una posición a la derecha en ambas listas con `.set()`.
 - o Insertá los valores en su lugar con `.set()`.

- Una función que **busque un alumno por nombre** → pedile un nombre al usuario. Usá `indexOf()` para encontrar su posición — si pos != -1, mostrará su nota. Si no existe, avisale.
- Una función que **muestre el ranking** → mostrará la lista ordenada numerada (1°, 2°, 3°...) con nombre y nota.
- Una función de **menú** → mostrará las opciones y llamará a la función correspondiente. Repetí hasta que el usuario elija Salir.

En `main()` creá las dos listas y llamará al menú.

💡 Métodos de `ArrayList` que vas a usar: `.add()`, `.get(i)`, `.set(i, valor)`, `.size()`, `.indexOf()`

Integración con Inteligencia Artificial

Los siguientes desafíos incorporan un modelo de lenguaje (LLM) para tomar decisiones sobre qué algoritmo usar, a partir de una consulta en lenguaje natural.

🤖 *Próximos desafíos — en construcción. ¡Viene lo mejor!*